

INFORME TÉCNICO PROFESIONAL: EVALUACIÓN Y PROPUESTAS DE MEJORA SOBRE CÓDIGO BASE

Análisis Crítico de Código Ajeno y Propuestas de Pull Requests para Práctica Profesional Supervisada

Proyecto:	Sistema Web de Gestión de Clientes (ProyectoLyEP2026)
Repositorio Auditado:	https://github.com/gustso/ProyectoLyEP2026
Ámbito / Cátedra:	Legislación y Ejercicio Profesional — Analista Programador Universitario (FI UNJu)
Rol del Auditor:	Arquitecto de Software & Tech Lead Senior
Fecha de Emisión:	Septiembre de 2026

COMPETENCIA PROFESIONAL CLAVE: LECTURA Y EVOLUCIÓN DE CÓDIGO HEREDADO

La habilidad más crítica de un Analista Programador no radica únicamente en escribir código desde cero, sino en comprender, evaluar rigurosamente y proponer mejoras sustanciales sobre lo que ya existe. El presente informe diagnostica la deuda técnica y los errores específicos del repositorio heredado, delimitando cuatro mejoras concretas de alto valor para ser distribuidas entre los integrantes del equipo mediante Pull Requests formalmente documentados.

1. ANÁLISIS TÉCNICO PROFESIONAL DEL CÓDIGO BASE

El proyecto auditado se presenta como una plataforma administrativa para la visualización, registro y administración de clientes comerciales, contando con vistas de autenticación, panel de control (Dashboard), listado con buscador reactivo, formulario de alta y vista detallada por cliente.

1.1 Diagnóstico de Arquitectura Real vs. Requerimientos

El primer resultado de la auditoría revela una marcada discrepancia arquitectónica entre las especificaciones asumidas y la realidad del código fuente:

- **A pesar de presumirse una arquitectura cliente-servidor completa (React + Node.js + Express), el repositorio aloja exclusivamente una Single Page Application (SPA) construida con React 19.2.6 y Vite 8.0.12. No existe implementación de servidor Express ni base de datos.**Inexistencia de la Capa de Backend:
- **La aplicación interactúa con la API de pruebas pública 'https://fakestoreapi.com/users'. Dicha API no persiste mutaciones reales (las peticiones POST y DELETE retornan un código 200/201 ficticio, pero los datos no quedan almacenados).**Persistencia Simulada de Solo Lectura:
- **El inicio de sesión y la asignación de roles de usuario (Soporte / Gerencia) se ejecutan síncronamente contra un arreglo estático en memoria dentro del frontend, sin emisión de tokens seguros (JWT), hashing de contraseñas ni control de sesiones.**Autenticación Ficticia en el Navegador:

1.2 Separación de Responsabilidades y Flujo de Datos

La estructura de directorios sigue una organización canónica para aplicaciones React (/components, /context, /hooks, /pages, /routes, /services, /css), pero adolece de inconsistencias de diseño:

- Existe 'clientesService.js' que emplea Axios para crear clientes, pero los componentes 'ListaClientes.jsx' y 'DetalleCliente.jsx' omiten este servicio y consumen la API mediante fetch() directo con URLs quemadas. Ruptura del Principio de Capa de Servicios:
- La sesión se administra en 'AutorizacionesContext' guardando el objeto 'admin' en localStorage, pero en paralelo 'Login.jsx' y 'DetalleCliente.jsx' escriben y leen una clave suelta 'role' directamente en localStorage, quebrando la reactividad del Context API. Doble Fuente de Verdad en Estado de Autenticación:
- En 'App.jsx', los componentes Header, Nav y Footer se renderizan de forma incondicional en todas las rutas, provocando que la barra de navegación se muestre incluso en la pantalla de Login. Jerarquía de Componentes Globales Inadecuada:

2. CATÁLOGO EXHAUSTIVO DE ERRORES PUNTUALES ENCONTRADOS

A continuación se detallan con precisión quirúrgica los errores más graves encontrados en el código base, indicando el archivo exacto, rango de líneas, código implicado y el riesgo técnico generado.

2.1 Error Puntual 1: Credenciales y Cuentas Hardcodeadas en el Bundle

Ubicación: Archivo: src/services/autorizacionesServices.js | Líneas: 1 a 38 | Severidad: CRÍTICA (Seguridad)

```
// src/services/autorizacionesServices.js
const usuarios = [
  { email: 'antonella@gmail.com', password: 'Admin123', nombre: 'Antonella', sector: 'Soporte' },
  { email: 'jimena@gmail.com', password: 'Admin123', nombre: 'Jimena', sector: 'Gerencia' },
  // ... 4 usuarios más con contraseñas en texto claro
]
```

Impacto Técnico: Vulnerabilidad CWE-798. Dado que React compila todo el código fuente del lado cliente, cualquier usuario puede abrir las herramientas de desarrollo del navegador (F12) o inspeccionar los archivos .js descargados y extraer la lista completa de correos, nombres y contraseñas de todos los operadores del sistema.

2.2 Error Puntual 2: Fuga de Contraseñas de Clientes en la Interfaz

Ubicación: Archivo: src/pages/DetalleCliente.jsx | Líneas: 86 a 94 | Severidad: ALTA (Privacidad/Seguridad)

```
// src/pages/DetalleCliente.jsx
<h2>Credenciales</h2>
<p><strong>Usuario:</strong> {cliente.username}</p>
<p><strong>Contraseña:</strong> {cliente.password}</p>
```

Impacto Técnico: Violación de confidencialidad de datos personales (CWE-312). La vista de detalle de un cliente expone abiertamente la contraseña en texto plano en la pantalla, un antipatrón inaceptable en sistemas corporativos donde los operadores jamás deben acceder a las contraseñas de los usuarios.

2.3 Error Puntual 3: Escalado de Privilegios por Control de Acceso Inseguro en Cliente

Ubicación: Archivo: src/pages/DetalleCliente.jsx | Líneas: 8 y 95 a 99 | Severidad: ALTA (Seguridad)

```
// src/pages/DetalleCliente.jsx
const role = localStorage.getItem('role');
// ...
```

```
{role?.trim() === 'Gerencia' && (  
  <button className='btn-eliminar' onClick={eliminarCliente}>Eliminar Cliente</button>  
)}
```

Impacto Técnico: Vulnerabilidad OWASP A01:2021 (Broken Access Control). La autorización para eliminar registros depende exclusivamente de una clave en localStorage leída directamente sin validación.

Cualquier usuario con rol 'Soporte' puede abrir la consola de JavaScript, ejecutar `localStorage.setItem('role', 'Gerencia')` y desbloquear de inmediato el botón de eliminación.

2.4 Error Puntual 4: Acceso Inseguro a Propiedades Anidadas sin Programación Defensiva

Ubicación: Archivo: `src/pages/ListaClientes.jsx` | Líneas: 30 a 38 | Severidad: ALTA (Estabilidad)

```
// src/pages/ListaClientes.jsx  
const clientesFiltrados = clientes.filter(  
  (cliente) =>  
    cliente.name.lastname.toLowerCase().includes(busqueda.toLowerCase()) ||  
    cliente.address.city.toLowerCase().includes(busqueda.toLowerCase())  
);
```

Impacto Técnico: Falta de programación defensiva. Si la API retorna un cliente donde el objeto 'name' o 'address' es nulo o indefinido (o si se crea un cliente nuevo con lastname vacío como ocurre en `FormCliente.jsx` línea 46), `cliente.name.lastname` arroja un error fatal en tiempo de ejecución: `'TypeError: Cannot read properties of undefined'`, destruyendo por completo la interfaz de la página.

2.5 Error Puntual 5: Dispersión y Violación de la Capa de Servicios

Ubicación: Archivos: `src/pages/ListaClientes.jsx` (13-28) y `src/pages/DetalleCliente.jsx` (14-27) | Severidad: MEDIA (Arquitectura)

```
// src/pages/ListaClientes.jsx  
useEffect(() => {  
  fetch('https://fakestoreapi.com/users')  
    .then((res) => { ... })  
}, []);
```

Impacto Técnico: Rompe la arquitectura en capas. Las vistas conocen la URL externa fija y la lógica de transporte HTTP nativo, mientras que en `FormCliente.jsx` se usa Axios a través de `clientesService.js`. Esto duplica lógica, impide centralizar interceptores de error y bloquea la posibilidad de cambiar de backend mediante variables de entorno (`.env`).

2.6 Error Puntual 6: Código Muerto y Conflicto en la Jerarquía de Rutas

Ubicación: Archivos: `src/routes/routes.jsx` (14-21) y `src/pages/Dashboard.jsx` (13-18) | Severidad: BAJA/MEDIA (Calidad)

```
// En routes.jsx la ruta '/' está envuelta en <RutaProtegida>  
// En Dashboard.jsx existe esta lógica:  
{!admin ? (  
  <div className='dashboard-login'>  
    <Login />  
  </div>  
) : ( ... )}
```

Impacto Técnico: Contradicción lógica y código muerto. `'RutaProtegida'` verifica si existe el usuario; si no existe, redirige de inmediato a `'/login'`. En consecuencia, la rama `'!admin'` en `Dashboar.jsx` jamás llegará a ejecutarse, generando confusión y acoplamiento innecesario entre `Dashboar` y `Login`.

2.7 Error Puntual 7: Desincronización Reactiva de Clientes

Ubicación: Archivos: `src/components/FormCliente.jsx` y `src/pages/ListaClientes.jsx` | Severidad: MEDIA (UX/Funcional)

Impacto Técnico: Cuando el operador completa el formulario 'Nuevo Cliente' en `FormCliente.jsx`, la petición se envía con éxito a la API simulada, pero la tabla de clientes ubicada en el mismo componente `ListaClientes.jsx` no se entera del cambio. No existe un estado compartido ni recarga reactiva, por lo que el cliente recién creado no aparece listado en pantalla a menos que se recargue el navegador manualmente.

2.8 Error Puntual 8: Métricas Cableadas y Antipatrones de Interacción

Ubicación: Archivos: `src/pages/Dashboard.jsx` (30-44) y `src/pages/Login.jsx` (48) | Severidad: MEDIA (Calidad)

Impacto Técnico: En el Dashboard las tarjetas de 'Clientes: 10', 'Gerencia: 3' y 'Soporte: 3' tienen números fijos quemados en el JSX que no reflejan la realidad de los datos. Adicionalmente, en `Login.jsx` se utiliza `'alert('Verifique los datos')'`, bloqueando el hilo de ejecución del navegador en lugar de utilizar componentes de retroalimentación asíncrona como `React-Bootstrap Alert`.

3. PROPUESTAS DE MEJORAS IMPLEMENTABLES (PULL REQUESTS DEL EQUIPO)

Para cumplir con la consigna de la Práctica Profesional ('al menos una mejora concreta por cada integrante, seleccionada del análisis, desarrollada e integrada al proyecto mediante un Pull Request documentado'), se formulan cuatro propuestas de alto valor técnico, perfectamente delimitadas y asignables individualmente:

3.1 Propuesta 1 (Integrante 1): Centralización de la Capa de Servicios API, Variables de Entorno y Programación Defensiva

- **Dispersión de llamadas HTTP, URLs quemadas en código y caídas de la interfaz por acceso no seguro a propiedades de clientes (Errores 2.4 y 2.5).** Problema Técnico Abordado:
- Archivos a Modificar / Crear:
 - [CREAR] `.env` y `.env.example`: Definir `VITE_API_BASE_URL=https://fakestoreapi.com`
 - [CREAR] `src/services/api.js`: Cliente Axios centralizado con `baseUrl` configurable y `timeout`.
 - [MODIFICAR] `src/services/clientesService.js`: Implementar los métodos `getClientes()`, `getClienteById(id)`, `crearCliente(data)` y `eliminarCliente(id)`.
 - [MODIFICAR] `src/pages/ListaClientes.jsx`: Reemplazar `fetch()` por `clientesService.getClientes()` e incorporar optional chaining (`cliente.name?.lastname ?? ''`) en el filtrado.
 - [MODIFICAR] `src/pages/DetalleCliente.jsx`: Reemplazar `fetch()` por `clientesService.getClienteById()` y `clientesService.eliminarCliente()`.
- **Aísla la infraestructura de red en un solo módulo, previene el 100% de los crashes por `TypeError` en el buscador y permite conectar la aplicación a un backend propio simplemente alterando una variable de entorno.** Impacto Técnico del PR:

3.2 Propuesta 2 (Integrante 2): Refactorización de Autenticación/RBAC, Saneamiento de Estado y Eliminación de Brechas de Seguridad

- **Credenciales expuestas en la vista, escalado de privilegios por manipulación de `localStorage` y código muerto en rutas protegidas (Errores 2.2, 2.3 y 2.6).** Problema Técnico Abordado:

- Archivos a Modificar / Crear:
 - [MODIFICAR] src/context/AutorizacionesContext.jsx: Unificar la sesión en un único estado 'admin', eliminando claves sueltas. Agregar la función utilitaria tieneRol(rolRequerido).
 - [MODIFICAR] src/pages/Login.jsx: Eliminar la escritura directa en localStorage.setItem('role', ...). Reemplazar alert() por el componente <Alert variant='danger'> de React-Bootstrap.
 - [MODIFICAR] src/pages/DetalleCliente.jsx: Eliminar del JSX la sección que muestra 'cliente.password'. Reemplazar la validación de role por const { tieneRol } = useAutorizaciones(); tieneRol('Gerencia').
 - [MODIFICAR] src/pages/Dashboard.jsx: Eliminar la importación de Login y la condición ternaria (!admin), dejando únicamente la visualización del panel administrativo.
- **Restaura una única fuente de verdad para la sesión del usuario, mitiga la vulnerabilidad de escalado de privilegios en el frontend, elimina fugas de contraseñas de clientes en pantalla y suprime código muerto en la navegación.** Impacto Técnico del PR:

3.3 Propuesta 3 (Integrante 3): Sincronización Reactiva de Clientes, Búsqueda Eficiente con Debounce y Estandarización de UX

- **Desincronización de clientes tras la creación, sobrecarga del hilo principal en el buscador y elementos visuales primitivos (Errores 2.7 y 2.8).** Problema Técnico Abordado:
- Archivos a Modificar / Crear:
 - [CREAR] src/hooks/useDebounce.js: Custom hook que retarde 300 ms la actualización del término de búsqueda para evitar filtrados innecesarios en cada tipeo.
 - [CREAR] src/context/ClientesContext.jsx (o refactor en hook de gestión): Proveer un estado global de clientes con la función agregarClienteLocal(nuevoCliente) que actualice la lista en memoria tras el POST exitoso.
 - [MODIFICAR] src/components/FormCliente.jsx: Al resolverse crearCliente, invocar agregarClienteLocal(respuesta) para refrescar la lista reactivamente sin recargar la página.
 - [MODIFICAR] src/pages/ListaClientes.jsx: Incorporar useDebounce en el input de búsqueda y reemplazar los títulos <h2>Cargando...</h2> por componentes <Spinner> y <Alert> de React-Bootstrap.
- **Optimiza el rendimiento del navegador en listas grandes, garantiza consistencia reactiva e inmediata en la interfaz y eleva sustancialmente la calidad visual y de experiencia de usuario (UX).** Impacto Técnico del PR:

3.4 Propuesta 4 (Integrante 4): Implementación de la Capa de Backend Base en Node.js y Express

- **Inexistencia de backend en el repositorio, simulación ficticia con FakeStoreAPI y almacenamiento de contraseñas de operadores en el frontend (Error 2.1 y Brecha Arquitectónica General).** Problema Técnico Abordado:
- Archivos a Modificar / Crear:
 - [CREAR] Directorio /server con package.json, integrando express, cors, dotenv, helmet y bcryptjs.
 - [CREAR] server/routes/authRoutes.js y server/controllers/authController.js: Endpoint POST /api/auth/login con verificación de credenciales mediante hash seguro (bcrypt) y firma de tokens JWT.
 - [CREAR] server/routes/clientesRoutes.js y server/controllers/clientesController.js: Endpoints CRUD reales (GET /api/clientes, POST /api/clientes, GET /api/clientes/:id, DELETE /api/clientes/:id) con persistencia estructurada.
 - [CREAR] server/middlewares/authMiddleware.js: Middleware para interceptar y verificar tokens de autorización en rutas protegidas y validar roles (Soporte vs. Gerencia).
 - [MODIFICAR] src/services/autorizacionesServices.js: Eliminar el array de usuarios en texto claro y consumir el nuevo endpoint POST /api/auth/login.

- **Convierte el prototipo en una solución de software cliente-servidor real y profesional, erradica la vulnerabilidad crítica de contraseñas quemadas y dota de persistencia de datos legítima al sistema.**Impacto Técnico del PR:

4. CONCLUSIÓN Y PLAN DE TRABAJO EN EQUIPO

El análisis exhaustivo confirma que el código existente representa una base de trabajo inicial con un diseño visual funcional, pero con severas carencias arquitectónicas y de seguridad. La puesta en marcha del proyecto por parte del equipo debe orientarse bajo un flujo de trabajo riguroso de integración continua:

- **Cada integrante creará un fork del repositorio, clonará localmente y configurará el upstream original.**Paso 1: Bifurcación y Preparación:
- **Cada integrante asumirá una de las cuatro propuestas detalladas en la Sección 3, trabajando en una rama temática aislada (e.g. feature/api-services, feature/auth-rbac, feature/clientes-debounce, feature/express-backend).**Paso 2: Asignación Individual de PR:
- **Se adoptarán convenciones de Conventional Commits (feat:, fix:, refactor:) y cada Pull Request incluirá descripción del problema, solución implementada y evidencias de prueba.**Paso 3: Commits Semánticos y Documentación:
- **Antes de fusionar cualquier rama, al menos dos integrantes del equipo deberán revisar y aprobar el Pull Request, garantizando la calidad y la transferencia de conocimiento entre pares.**Paso 4: Code Review Cruzado:

Propuesta / PR	Responsable Sugerido	Archivos Clave	Objetivo Principal
PR #01: Servicios & API	Integrante 1	api.js, clientesService.js, ListaClientes.jsx	Centralizar HTTP, variables .env y evitar caídas por null
PR #02: Auth & Seguridad	Integrante 2	AutorizacionesContext, DetalleCliente, Dashboard	Unificar RBAC, ocultar contraseñas y limpiar código muerto
PR #03: Reactividad & UX	Integrante 3	ClientesContext, useDebounce, FormCliente	Sincronizar altas reactivamente y optimizar buscador
PR #04: Backend Express	Integrante 4	server/, routes/, controllers/, authMiddleware	Crear API REST real y erradicar contraseñas del cliente